Assigment 3 : Image change captioning

November 25, 2025

1 Task overview:

- In this work, you are provided with an image captioning model with several part missing, and you are required to complement it, then train it on your own.
- The definition of image captioning mode is : Given two images, the model is required to output a caption describing the difference between the two images.

2 Method

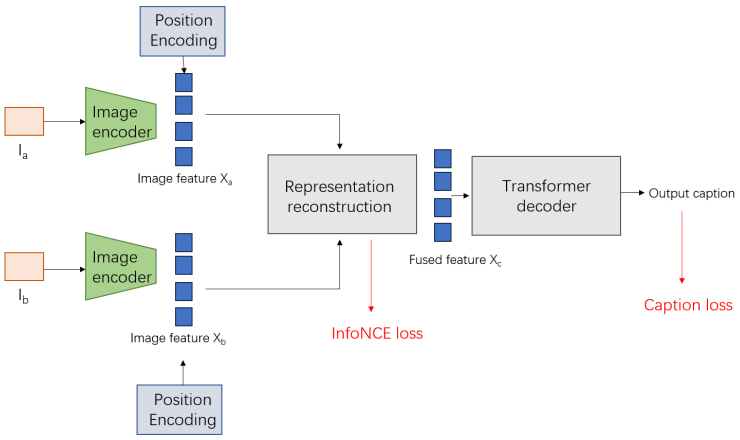

Figure 1: Method Overview.

The overall architecture is shown in Fig. 1. The method contains following steps:

2.1 Input Image Encoding

- Given paired input images I_b and I_a, we utilize image encoder, which is a pre-trained CNN, to extract their features, denoted as X_a and X_b, where $X \in R^{C \times H \times W}$. C, H, W indicate the number of channels, height, and width.
- Project feature into low-dimensional embedding space of R^D:

$$\tilde{X}_o = conv_2(X_o) + pos(X_o), \tag{1}$$

where $o \in (a, b)$. $conv_2$ denotes a 2D-convolutional layer; pos is a learnable position embedding layer.

2.2 Token-wise Matching for representation learning.

- Introduce Token-wise Matching for representation learning.
- For token matching, given query $Q \in R^{N \times D}$ and key $K \in R^{N \times D}$, compute the similarity of i-th query token with all key tokens and select the maximum one as its token-wise maximum similarity with K.

$$TM(Q,K) = \left[\frac{1}{N} \sum_{i=1}^{N} \max_{j=1}^{N} \left(e_{i,j}\right) + \frac{1}{N} \sum_{j=1}^{N} \max_{i=1}^{N} \left(e_{i,j}\right) \right] /2, e_{i,j} = \left(q_i\right)^{\top} k_j. \tag{2}$$

- Then extend it into multi-head version:

$$MTM(Q,K) = Concat_{i'=1\ldots h}\left(head_{i'}\right), head_{i'} = TM\left(QW_{i'}^{Q}, KW_{i'}^{K}\right). \tag{3}$$

- For a batch of B paired images, reshape $\tilde{X} \in R^{D \times H \times W}$ to $\tilde{X} \in R^{N \times D}$. Then use MTM to compute similarity ($B \times B$ matrix).
- Use InfoNCE loss to maximize cross-view contrastive alignment between $\tilde{X}_k^b$ and $\tilde{X}_k^a$ while minimizing the alignment of non-similar images.

$$\mathcal{L}_{b2a} = -\frac{1}{B} \sum_{k}^{B} \log \frac{\exp\left(MTM\left(\tilde{X}_k^b, \tilde{X}_k^a\right)/\tau\right)}{\sum_r^B \exp\left(MTM\left(\tilde{X}_k^b, \tilde{X}_r^a\right)/\tau\right)} \tag{4}$$

$$\mathcal{L}_{a2b} = -\frac{1}{B} \sum_{k}^{B} \log \frac{\exp\left(MTM\left(\tilde{X}_k^a, \tilde{X}_k^b\right)/\tau\right)}{\sum_r^B \exp\left(MTM\left(\tilde{X}_k^a, \tilde{X}_r^b\right)/\tau\right)} \tag{5}$$

$$\mathcal{L}_{cv} = \frac{1}{2}(\mathcal{L}_{b2a} + \mathcal{L}_{a2b}), \tag{6}$$

2.3 Representation Reconstruction

- Use multi-head cross-attention (MHCA) to reconstruct the unchanged representations for each image:

$$\tilde{X}_b^u = MHCA(\tilde{X}_b, \tilde{X}_a, \tilde{X}_a), \tilde{X}_a^u = MHCA(\tilde{X}_a, \tilde{X}_b, \tilde{X}_b). \tag{7}$$

- Feature integration with LayerNorm, FFN and ReLU:

$$\tilde{X}_o^c = LN(\tilde{X}_o + \tilde{X}_o^u). \tag{8}$$

$o \in (b, a)$

$$\tilde{X}_c = ReLU\left(\left[\tilde{X}_b^c; \tilde{X}_a^c\right] W_h + b_h\right), \tag{9}$$

[;] is a concatenation operation.

2.4 Caption Generation

- Use transformer decoder to translate $\tilde{X}_c \in R^{N \times D}$ into a sentence.
- Use GT word features $E[W] = \{E[w_1], ..., E[w_m]\}$ to query the most related features $\hat{H}$ from $\tilde{X}_c$ via the multi-head cross-attention, then turn it into probability.

$$\tilde{W} = Softmax\left(\tilde{H} W_c + b_c\right), \tag{10}$$

2.5 Training

Loss contains two parts, caption loss and InfoNCEloss.

- Given GT words $(w_1^*, \ldots, w_m^*)$, we minimize the negative log-likelihood loss:

$$\mathcal{L}_{cap}(\theta) = -\sum_{t=1}^{m} \log p_\theta\left(w_t^* \mid w_{<t}^*\right), \tag{11}$$

where $p_\theta\left(w_t^* \mid w_{<t}^*\right)$ is computed by Eq. (10), and θ are the parameters of the network.

-

$$\mathcal{L} = \mathcal{L}_{cap} + \lambda_v \mathcal{L}_{cv} \tag{12}$$

3 Setup

- The code is tested on Linux + NVIDIA GeForce RTX 3090
- Create a virtual environment (python 3.8 is recommended)
- Install required packages (pip install -r requirements.txt)
- Download en_core_web_sm for spacy from LINK
- Install evaluation tools following LINK

4 Data preparation

- Download dataset from LINK and unzip it under ./data folder
- Process data with following steps:

```
python scripts/extract_features.py --input_image_dir
↪  ./data/images --output_dir ./data/features

python scripts/extract_features.py --input_image_dir
↪  ./data/sc_images --output_dir ./data/sc_features

python scripts/extract_features.py --input_image_dir
↪  ./data/nsc_images --output_dir ./data/nsc_features
```

- Build vocabs

```
python scripts/preprocess_captions_transformer.py
```

- modify annotation files

```
    mv total_change_captions_reformat_subset.json ./data

    mv splits_subset.json ./data
```

5 Code to Implement

5.1 Position Encoding

- Implement the code at models/transformer_decoder.py Line 26

5.2 InfoNCE loss

- Implement the code at utils/utils.py Line 296

5.3 Attention Mechanisms.

- Self attention: Implement the code at models/transformer_decoder.py Line 109
- Cross attention: Implement the code at models/transformer_decoder.py Line 152
- Attention: Implement the code at models/model.py Line 35

6 Training

```
python train.py
```

Note that for model training, you can modify the config file in configs/dynamic/transformer_fast.yaml(such as the training steps) on your own.

7 Inference

```
python test.py
```

8 Evaluation

```
python evaluate.py
```

9 Handin Files

You need to return the following files:

- models/transformer_decoder.py
- models/model.py
- utils/utils.py
- evaluation results on testset.(can be a screenshot photo, which will be saved in test_output/captions/eval_results.txt after excuting evaluation)
- A visualization result of your own trained model, including the input two images and the output caption.
- Bonus: you can find an advancved VLM to accomplish the image change captioning task, and discuss the advantages VLM might have.